

选择的艺术

条件判断与分支结构 · 14题 · C++ & Python 双语对照

第二周，小鲁已经能熟练地写A+B了。但小华说："真正的程序不会只是直线执行——你根据不同的情况做出不同的决定。如果成绩及格就通过，否则补考；如果是周一就升旗，否则早读……这就是条件判断。"

本章14道题，小鲁将从简单的if-else开始，逐步掌握多分支、嵌套条件、决策树和dict映射表。最重要的是学会"用数据结构替代逻辑堆叠"——当你看到一大串if-elif-else的时候，第一反应应该问自己："能不能用一张表来替代？"

C++ vs Python 条件判断速查

- 单分支: `if(cond){...}` → `if cond:...`
- 多分支: `if/else if/else` → `if/elif/else`
- 链式比较: `0<=x&&x<=25` → `0<=x<=25` (Python独有!)
- 映射表: 8个if-else → `dict.get(key,default)` (Python独有!)
- 三元表达式: `cond?a:b` → `a if cond else b`

NQ015 倍数 AcWing 665

小鲁在数学课上学了倍数。"如果A能被B整除，或者B能被A整除，它们就是倍数关系。"小华说："编程里用%取模来判断——`a%b==0`说明a是b的倍数。用`||`(C++)或`or`(Python)把两个条件连起来。"

输入 一行，两个整数A和B。

输出 互为倍数输出"`Sao Multiplos`"，否则"`Nao sao Multiplos`"。

范围 $-10^4 \leq A, B \leq 10^4$ ($A, B \neq 0$)

输入

6 24

输出

Sao Multiplos

思路 用取模判断倍数关系：`A%B==0`或`B%A==0`。两个条件用"或"连接。这是if-else最简单的形式——单一条件判断，两个分支。

技巧 Py的if语句不需要括号。C++的%运算符要求整数。`a%b==0`比`!(a%b)`更易读。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int a, b;
    cin >> a >> b;

    if (a % b == 0 || b % a == 0) cout << "Sao Multiplos" << endl;
    else cout << "Nao sao Multiplos" << endl;

    return 0;
}
```

Python

```
a, b = map(int, input().split())
if a % b == 0 or b % a == 0:
    print("Sao Multiplos")
else:
    print("Nao sao Multiplos")
```

NQ016 零食 AcWing 660

小鲁去小卖部买零食：1号零食4元，2号4.5元，3号5元，4号2元，5号1.5元。"这么多选项，怎么写？"小华说："C++可以用if-else链或switch。Python可以用dict映射——一行搞定全部！"

输入

一行，两个整数：零食编号X和数量Y。

输出

"Total: R\$ "后跟总价（保留2位）。

范围

$1 \leq X \leq 5, 1 \leq Y \leq 100$ 。单价：1=4.00, 2=4.50, 3=5.00, 4=2.00, 5=1.50

输入

3 2

输出

Total: R\$ 10.00

思路 多分支选择。C++用if-else链或switch，Python用if-elif或dict映射。本课重点：dict={1:4.0,2:4.5,3:5.0,4:2.0,5:1.5}，prices[x]*y一行出结果——O(1)查找，代码量减少50%。

技巧 Py的dict映射表：prices={1:4.0,2:4.5,3:5.0,4:2.0,5:1.5}。替代if-elif链。"用数据驱动替代逻辑堆叠"是进阶编程的核心理念。

C++

```
#include <cstdio>

int main()
{
    int x, y;
    scanf("%d%d", &x, &y);

    double price;
    if (x == 1) price = 4;
    else if (x == 2) price = 4.5;
    else if (x == 3) price = 5;
    else if (x == 4) price = 2;
    else price = 1.5;

    printf("Total: R$ %.2lf\n", price * y);

    return 0;
}
```

Python

```
prices = {1: 4.0, 2: 4.5, 3: 5.0, 4: 2.0, 5: 1.5}
x, y = map(int, input().split())
print(f"Total: R$ {prices[x] * y:.2f}")
```

NQ017 区间 AcWing 659

"一个数落在哪个区间里？"小鲁看着题目："0到25是闭区间，25到50是左开右闭....."小华提醒他："注意开闭区间的区别——[0,25]包含25，(25,50]不包含25但包含50。Python的链式比较 $0 \leq x \leq 25$ 比C++的 $0 \leq x \&\& x \leq 25$ 更直观。"

输入

一个浮点数x。

输出

区间名或"Fora de intervalo"。

范围

 $-10^9 \leq x \leq 10^9$

输入

25.01

输出

Intervalo (25,50]

思路 从最小区间开始判断，逐级向上。Python链式比较更直观，C++需分开写 $\&\&$ 。注意开闭括号[包含](不包含)的数学含义——这是区间判断最容易出错的地方。

技巧 Python链式比较： $0 \leq x \leq 25$ 。C++需要 $0 \leq x \&\& x \leq 25$ 。Python的链式比较是语法糖——编译器自动转换成两个比较的and。

C++

```
#include <iostream>

using namespace std;

int main()
{
    double x;
    cin >> x;

    if (x >= 0 && x <= 25) cout << "Intervalo [0,25]" << endl;
    else if (x > 25 && x <= 50) cout << "Intervalo (25,50]" << endl;
    else if (x > 50 && x <= 75) cout << "Intervalo (50,75]" << endl;
    else if (x > 75 && x <= 100) cout << "Intervalo (75,100]" << endl;
    else cout << "Fora de intervalo" << endl;

    return 0;
}
```

Python

```
x = float(input())
if 0 <= x <= 25:
    print("Intervalo [0,25]")
elif 25 < x <= 50:
    print("Intervalo (25,50]")
elif 50 < x <= 75:
    print("Intervalo (50,75]")
elif 75 < x <= 100:
    print("Intervalo (75,100]")
else:
    print("Fora de intervalo")
```

NQ018 三角形 AcWing 664

"给你三根木棍，长度分别是6.0、4.0和2.0——能拼成三角形吗？"小栋问。小鲁想了想："三角形的条件是任意两边之和大于第三边。2+4=6，等于第三边……不能！"小华说："必须同时满足 $A+B>C$ 、 $A+C>B$ 、 $B+C>A$ 三个条件，用&&连接。"

输入 一行，三个浮点数A、B、C。

输出 能构成则"Perimetro = 周长"，否则"Area = 梯形面积"。

范围 0

输入

6.0 4.0 2.0

输出

Area = 10.0

思路 三角形判定： $A+B>C$ 且 $A+C>B$ 且 $B+C>A$ 。三个条件必须同时满足，用&&(C++)或and(Python)连接。不能构成时输出梯形面积 $(A+B)*C/2$ 。

技巧 三个条件同时满足：if $a+b>c$ and $a+c>b$ and $b+c>a$ 。Python可以写得非常干净。注意浮点数比较的精度问题——对于大数或极端接近的情况需谨慎。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    double a, b, c;
    cin >> a >> b >> c;
    if(a + b > c && a + c > b && b + c > a)
        printf("Perimetro = %.1lf\n", a + b + c);
    else
        printf("Area = %.1lf\n", (a + b) * c / 2);

    return 0;
}
```

Python

```
a, b, c = map(float, input().split())
if a + b > c and a + c > b and b + c > a:
    print(f"Perimetro = {a + b + c:.1f}")
else:
    print(f"Area = {(a + b) * c / 2:.1f}")
```

NQ019 游戏时间 AcWing 667

小鲁玩游戏从下午4点玩到凌晨2点。"这跨天了，怎么算？"小华说："A=16,B=2。如果A

输入

两个整数A和B ($0 \leq A, B \leq 23$)。

输出

"O JOGO DUROU X HORA(S)".

范围

$0 \leq A, B \leq 23$

输入

16 2

输出

0 JOGO DUROU 10 HORA(S)

思路 先判断是否跨天：A

技巧 可以统一为 $(B - A + 24) \% 24$ ，但结果为0时需要特殊处理（表示24小时）。这种"循环型"问题在时间/角度/周期问题中反复出现。

C++

```
#include <cstdio>

int main()
{
    int a, b;
    scanf("%d%d", &a, &b);

    int res;
    if (a < b) res = b - a;
    else res = b - a + 24;

    printf("O JOGO DUROU %d HORA(S)\n", res);

    return 0;
}
```

Python

```
a, b = map(int, input().split())
if a < b:
    d = b - a
else:
    d = b - a + 24
print(f"O JOGO DUROU {d} HORA(S)")
```

NQ020 加薪 AcWing 669

小鲁收到年终调薪通知——涨多少取决于当前工资：0–400涨15%，400.01–800涨12%，800.01–1200涨10%，1200.01–2000涨7%，2000以上涨4%。"这是分段函数！"小华说："从低到高判断最自然—— $s \leq 400?$ $15:s \leq 800?12:s \leq 1200?10:...$ "

- 输入** 一个浮点数表示当前工资。
- 输出** 三行：新工资、涨薪金额、涨幅百分比。
- 范围** $0 < \text{工资} \leq 10^6$

输入

400.00

输出

Novo salario: 460.00
Reajuste ganho: 60.00
Em percentual: 15 %

思路 从低到高的if-elif链。区间互斥，顺序很重要——先判断小值再判断大值可以不加&&上限条件（因为前面的判断已经排除了）。注意输出格式：保留2位小数+空格+%。

技巧 C++中%%输出%字样（printf需要转义）。Python直接写%。区间判断顺序从低到高可以利用else的"否则"语义简化条件。

C++

```
#include <cstdio>

int main()
{
    double salary;
    scanf("%lf", &salary);

    double x;

    if (salary <= 400) x = 0.15;
    else if (salary <= 800) x = 0.12;
    else if (salary <= 1200) x = 0.1;
    else if (salary <= 2000) x = 0.07;
    else x = 0.04;

    printf("Novo salario: %.2lf\n", salary * (1 + x));
    printf("Reajuste ganho: %.2lf\n", salary * x);
    printf("Em percentual: %.0lf %%", x * 100);

    return 0;
}
```

Python

```
s = float(input())
if s <= 400: p = 15
elif s <= 800: p = 12
elif s <= 1200: p = 10
elif s <= 2000: p = 7
else: p = 4
r = s * p / 100
print(f"Novo salario: {s + r:.2f}")
print(f"Reajuste ganho: {r:.2f}")
print(f"Em percentual: {p} %")
```

NQ021 动物 AcWing 670

"猜动物游戏！"小鲁兴奋地说。"输入三个特征——脊椎/非脊椎、哺乳/鸟/昆虫/环节动物、食性——判断是什么动物。"小华说："这是嵌套if的完美案例——像一棵决策树，每个内部节点是判断，叶子节点是答案。"

输入 三行字符串（vertebrado/invertebrado等）。

输出 动物名称（aguia/pomba/homem/vaca/pulga/lagarta/sanguessuga/minhoca）。

范围 输入保证合法。

输入	输出
vertebrado	homem
mamifero	
onivoro	

思路 三层嵌套if。先判断脊椎/非脊椎→再判断纲→再判断食性。本质是一棵深度为3的决策树。Python的嵌套if-elif结构清晰易读。也可用dict嵌套：tree={'vertebrado':{'ave':{'...}}}。

技巧 Python的嵌套dict可以替代嵌套if——tree[a][b][c]一行定位答案。但写if逻辑更透明，两者都是合理的方案。"数据结构化思维"是编程进阶的关键能力。

C++

```
if (a == "vertebrado")
{
    if (b == "ave")
    {
        if (c == "carnivoro") cout << "aguia" << endl;
        else cout << "pomba" << endl;
    }
    else
    {
        if (c == "onivoro") cout << "homem" << endl;
        else cout << "vaca" << endl;
    }
}
else
{
    if (b == "inseto")
    {
        if (c == "hematofago") cout << "pulga" << endl;
        else cout << "lagarta" << endl;
    }
    else
    {
        if (c == "hematofago") cout << "sanguessuga" << endl;
        else cout << "minhoca" << endl;
    }
}
```

Python

```
a, b, c = input(), input(), input()
if a == "vertebrado":
    if b == "ave":
        print("aguia" if c == "carnivoro" else "pomba")
    else:
        print("homem" if c == "onivoro" else "vaca")
else:
    if b == "inseto":
        print("pulga" if c == "hematofago" else "lagarta")
    else:
        print("sanguessuga" if c == "hematofago" else "minhoca")
```

NQ022 选择练习1 AcWing 657

小鲁一口气写下五个条件：B>C、D>A、C+D>A+B、C>0、D>0、A是偶数。"要同时满足才能输出accepted——用&&(and)把它们串起来。"小华说："这是复杂布尔表达式的练习——五个条件，一个都不能少。"

- 输入** 一行四个整数A、B、C、D。
- 输出** 全部满足输出"Valores aceitos", 否则"Valores nao aceitos"。
- 范围** $-10^9 \leq A, B, C, D \leq 10^9$

输入

5 6 7 8

输出

Valores nao aceitos

思路 5个条件用&&(and)连接。同时满足才输出accepted。训练复杂布尔表达式的使用。条件的排列顺序影响短路求值——先放最容易判断的。

技巧 Python把5个条件写一行: if b>c and d>a and c+d>a+b and c>0 and d>0 and a%2==0。可读性极好。先放C>0、D>0这种简单条件可以触发短路优化。

C++

```
if(b > c && d > a && c + d > a + b && c > 0 && d > 0 && a % 2 == 0)
    cout << "Valores aceitos" << endl;
else
    cout << "Valores nao aceitos" << endl;

return 0;
```

Python

```
a, b, c, d = map(int, input().split())
if b > c and d > a and c + d > a + b
and c > 0 and d > 0 and a % 2 == 0:
    print("Valores aceitos")
else:
    print("Valores nao aceitos")
```

NQ023 DDD AcWing 671

小鲁想给其他城市的同学打电话。"61是巴西利亚，71是萨尔瓦多，11是圣保罗……这么多城市怎么记？"小华说："C++可以写8个if-else。但Python用dict——{'61':'Brasilia','71':'Salvador',...}，一行get()全搞定。这就是'用数据驱动替代代码堆叠'的力量。"

- 输入** 一个整数DDD区号。
- 输出** 城市名或"DDD nao cadastrado"。
- 范围** DDD为正整数。

输入

11

输出

Sao Paulo

思路 Python的dict映射表一行替代8个if-else。ddd.get(ddd, 'DDD nao cadastrado')——查找存在就返回，不存在就返回默认值。这是本课最重要的工程思维：看到重复的if-else模式时，用数据结构替代。

技巧 ddd={'61':'Brasilia','71':'Salvador','11':'Sao Paulo','21':'Rio de Janeiro','32':'Juiz de Fora','19':'Campinas','27':'Vitoria','31':'Belo Horizonte'}。print(ddd.get(x,'DDD nao cadastrado'))。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int x;
    cin >> x;
    if (x == 61) cout << "Brasilia" << endl;
    else if (x == 71) cout << "Salvador" << endl;
    else if (x == 11) cout << "Sao Paulo" << endl;
    else if (x == 21) cout << "Rio de Janeiro" << endl;
    else if (x == 32) cout << "Juiz de Fora" << endl;
    else if (x == 19) cout << "Campinas" << endl;
    else if (x == 27) cout << "Vitoria" << endl;
    else if (x == 31) cout << "Belo Horizonte" << endl;
    else cout << "DDD nao cadastrado" << endl;

    return 0;
}
```

Python

```
ddd = {61:"Brasilia",71:"Salvador",11:"Sao Paulo",21:"Rio de Janeiro",32:"Juiz de Fora",19:"Campinas",27:"Vitoria",31:"Belo Horizonte"}
x = int(input())
print(ddd.get(x, "DDD nao cadastrado"))
```

NQ024 点的坐标 AcWing 662

小鲁在坐标系上画了一个点：P(4.5,-2.2)——这是第几象限？""第四象限！"小华说，"但要注意特殊情况：原点(0,0)、X轴(y=0)、Y轴(x=0)。判断顺序很重要——先判断原点，再判断轴，最后判断象限。如果顺序搞错——比如先判断x>0就归为Q1——可能把原点误判。"

输入

两个浮点数x和y。

输出

Q1/Q2/Q3/Q4/Origem/Eixo X/Eixo Y。

范围

 $-10^9 \leq x, y \leq 10^9$

输入

4.5 -2.2

输出

Q4

思路 判断顺序：原点→轴→象限。这是分类讨论的标准流程——先特殊后一般。x>0且y>0→Q1，x<0且y>0→Q2，x<0且y<0→Q3，x>0且y<0→Q4。

技巧 判断顺序从特殊到一般：原点→轴→象限。Python中x==0和y==0用浮点比较在本题数据范围内安全。对于更精确的场景可使用abs(x)<1e-9。

C++

```
#include <iostream>

using namespace std;

int main()
{
    double x, y;
    cin >> x >> y;

    if (x > 0 && y > 0) cout << "Q1" << endl;
    else if (x < 0 && y > 0) cout << "Q2" << endl;
    else if (x < 0 && y < 0) cout << "Q3" << endl;
    else if (x > 0 && y < 0) cout << "Q4" << endl;
    else
    {
        if (!x && !y) cout << "Origem" << endl;
        else if (!y) cout << "Eixo X" << endl;
        else cout << "Eixo Y" << endl;
    }
    return 0;
}
```

Python

```
x, y = map(float, input().split())
if x > 0 and y > 0: print("Q1")
elif x < 0 and y > 0: print("Q2")
elif x < 0 and y < 0: print("Q3")
elif x > 0 and y < 0: print("Q4")
elif x == 0 and y == 0: print("Origem")
elif x == 0: print("Eixo Y")
else: print("Eixo X")
```

NQ025 三角形类型 AcWing 666

"不仅要判定能不能构成三角形，还要分直/钝/锐/等边/等腰——这一道题等于前一道的10倍工作量！"小鲁被吓到了。小华笑道："别怕，先把三边降序排列 ($a \geq b \geq c$)，然后从一般到特殊逐个判断。排序降序是简化三角形分类的关键——排完序后一切都变简单了。"

输入

三个浮点数A、B、C。

输出

按顺序判断：不构成→直角→钝角→锐角→等边→等腰。可能输出多行。

范围

0

输入

7.0 5.0 7.0

输出

TRIANGULO ACUTANGULO
TRIANGULO ISOSCELES

思路 三步法：1.降序排列(swap直到 $a \geq b \geq c$)；2.判三角形： $a \geq b+c$?不能构成；3.角度类型： $a^2=b^2+c^2$ (直角)、 $a^2 > b^2+c^2$ (钝角)、 $a^2 < b^2+c^2$ (锐角)。

技巧 Python一行排序：`a,b,c=sorted([a,b,c],reverse=True)`。排序降序后所有判断条件都变得简单。注意用多个if而非if-elif——因为可能同时满足多个条件（如既是锐角又是等腰）。

C++

```
#include <iostream>

using namespace std;

int main()
{
    double a, b, c;
    cin >> a >> b >> c;

    if (b > a)
    {
        double t = a;
        a = b;
        b = t;
    }
    if (c > a)
    {
        double t = a;
        a = c;
        c = t;
    }
    if (c > b)
    {
        double t = b;
        b = c;
        c = t;
    }

    if (a >= b + c) cout << "NAO FORMA TRIANGULO" << endl;
    else
    {
        if (a * a == b * b + c * c) cout << "TRIANGULO RETANGULO" << endl;
        if (a * a > b * b + c * c) cout << "TRIANGULO OBTUSANGULO" << endl;
        if (a * a < b * b + c * c) cout << "TRIANGULO ACUTANGULO" << endl;
        if (a == b && b == c) cout << "TRIANGULO EQUILATERO" << endl;
        else if (a == b || a == c || b == c) cout << "TRIANGULO ISOSCELES" << endl;
    }

    return 0;
}
```

Python

```
a, b, c = sorted(map(float, input().split()), reverse=True)
if a >= b + c:
    print("NAO FORMA TRIANGULO")
else:
    if a*a == b*b + c*c: print("TRIANGULO RETANGULO")
    if a*a > b*b + c*c: print("TRIANGULO OBTUSANGULO")
    if a*a < b*b + c*c: print("TRIANGULO ACUTANGULO")
    if a == b == c: print("TRIANGULO EQUILATERO")
    elif a == b or b == c or a == c:
        print("TRIANGULO ISOSCELES")
```

NQ026 游戏时间2 AcWing 668

"上次只算了小时，这次要精确到分钟！"小鲁升级了游戏计时器。"起点7:08，终点9:10——持续多久？"统一换算成分钟：7*60+8=428，9*60+10=550，差122分钟=2小时2分钟。如果终点≤起点，加24*60处理跨天。"

输入 四个整数：A(开始时)、B(开始分)、C(结束时)、D(结束分)。

输出 "O JOGO DUROU X HORA(S) E Y MINUTO(S)"。

范围 0≤A,C≤23,0≤B,D≤59

输入

7 8 9 10

输出

0 JOGO DUROU 2 HORA(S) E 2 MINUTO(S)

思路 统一转换为分钟：start=A*60+B,end=C*60+D。若end≤start则加24*60。diff=end-start，再转回时=diff/60,分=diff%60。"统一量纲"是时间问题的通用技巧。

技巧 统一量纲(分钟)后比较和计算避免分别处理小时和分钟的复杂情况。Python的divmod(diff,60)可以直接分解时和分。

C++

```
#include <stdio>

int main()
{
    int a, b, c, d;
    scanf("%d%d%d%d", &a, &b, &c, &d);

    int start = a * 60 + b;
    int end = c * 60 + d;

    int spent_time = end - start;
    if (spent_time <= 0) spent_time += 1440;

    printf("0 JOGO DUROU %d HORA(S) E %d MINUTO(S)", spent_time / 60, spent_time % 60);

    return 0;
}
```

Python

```
a, b, c, d = map(int, input().split())
s = a * 60 + b
e = c * 60 + d
if e <= s: e += 24 * 60
d = e - s
print(f"0 JOGO DUROU {d // 60} HORA(S) E {d % 60} MINUTO(S)")
```

NQ027 税 AcWing 672

小鲁第一次交税。"税率分段计算——0到2000免税，2000.01到3000收8%，3000.01到4500收18%，4500以上收28%。""注意是分段计税，不是全额×最高税率！"小华强调："每一段分别计算——超过2000的部分才需要交税。"

输入

一个浮点数表示月收入。

输出

免税输出"Isento"，否则"R\$ XX.XX"。

范围

0<收入≤10⁶

输入

3002.00

输出

R\$ 80.36

思路 分段计税：先判断是否≤2000(免税)。超出2000部分逐段按税率计算。Python用min()限制每段上限：tax=min(x,1000)*0.08+min(max(x-1000,0),1500)*0.18+...

技巧 Python的min/max组合：min(x,1000)*0.08处理第一段(0–1000按8%)。理解“分段”原理比写出代码更重要——这是个人所得税的核心逻辑。

C++

```
#include <cstdio>

int main()
{
    double x;
    scanf("%lf", &x);

    double sum = 0;
    if (x > 2000)
    {
        double y = 3000;
        if (x < 3000) y = x;
        sum += (y - 2000) * 0.08;
    }
    if (x > 3000)
    {
        double y = 4500;
        if (x < 4500) y = x;
        sum += (y - 3000) * 0.18;
    }
    if (x > 4500) sum += (x - 4500) * 0.28;

    if (sum == 0) printf("Isento");
    else printf("R$ %.2lf\n", sum);

    return 0;
}
```

Python

```
x = float(input())
if x <= 2000:
    print("Isento")
else:
    x -= 2000
    t = 0
    if x > 0:
        v = min(x, 1000)
        t += v * 0.08
        x -= v
    if x > 0:
        v = min(x, 1500)
        t += v * 0.18
        x -= v
    if x > 0:
        t += x * 0.28
    print(f"R$ {t:.2f}")
```

NQ028 简单排序 AcWing 663

"把三个数从小到大排列——这是排序算法的萌芽。"小华说："三步比较交换：先比较a和b，必要时交换；再比较a和c；最后比较b和c。三步之后 $a \leq b \leq c$ ——这就是冒泡排序思想的微型版。C++用swap(), Python用a,b=b,a——交换变量最优雅的方式。"

输入

一行三个整数。

输出

升序排列的三个数。

范围

$-10^9 \leq a, b, c \leq 10^9$

输入

7 21 -14

输出

-14 7 21

思路 三步交换排序：if a>b:swap(a,b); if a>c:swap(a,c); if b>c:swap(b,c)。这是排序算法的最小实现——理解交换操作的原理为后续学习排序算法打下基础。

技巧 Python的a,b=b,a是最优雅交换写法——基于元组解包。C++用std::swap或临时变量。Python一行sorted()也行但手写交换理解原理更重要。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int a, b, c;
    cin >> a >> b >> c;

    int x = a, y = b, z = c;

    if (b < a)
    {
        int t = a;
        a = b;
        b = t;
    }
    if (c < a)
    {
        int t = a;
        a = c;
        c = t;
    }
    if (c < b)
    {
        int t = b;
        b = c;
        c = t;
    }

    cout << a << endl << b << endl << c << endl << endl;
    cout << x << endl << y << endl << z << endl;

    return 0;
}
```

Python

```
a, b, c = map(int, input().split())
if a > b: a, b = b, a
if a > c: a, c = c, a
if b > c: b, c = c, b
print(a, b, c)
```

本章知识点总结

- **if-else/if-elif-else**：条件分支的基本语法，两语言核心差异在于Python的elif和缩进
- **Python链式比较**： $a \leq x \leq b$ 等价于 $a \leq x$ and $x \leq b$ ，语法糖但可读性极好
- **dict映射表**：替代长if-else链的利器。O(1)查找，代码减少50%+
- **决策树思维**：嵌套条件判断本质是决策树——从根到叶的路径
- **排序简化**：三角形类型题——先排序使 $a \geq b \geq c$ ，后续所有判断条件简化
- **分段计算**：税务题——min/max组合处理分段，理解"分段"而非"全额"